

★ Complete Interview + Backend Guide ★

1. What is Node.js?

Node.js is a **JavaScript runtime** built on **Chrome's V8 engine**. It allows us to run JavaScript on the **server side** and build fast, scalable network applications.

2. Why Node.js?

- ✓ JavaScript everywhere (Frontend + Backend)
- ✓ Fast execution (**V8 Engine**)
- ✓ Non-blocking I/O model
- ✓ Event-driven & lightweight
- ✓ Huge ecosystem (**NPM**)
- ✓ Perfect for real-time apps & APIs

3. V8 Engine

V8 is **Google's** open-source JavaScript engine. It compiles JavaScript directly into machine code which makes Node.js extremely fast.

V8

4. Runtime vs Framework

Runtime (Node.js)	Framework (Express.js)
Provides environment to run JavaScript. Focuses on executing code. Example: Node.js	Provides structure & tools to build apps. Built on top of Node.js Example: Express.js

5. Node.js Architecture

- Built on **event-driven, non-blocking I/O** model
- Uses a **single thread** with an event loop
- Handles multiple connections concurrently
- Highly **scalable & efficient**

6. Single Threaded but Non-Blocking

Node.js uses a **single thread** to handle multiple requests using **non-blocking I/O** operations. It offloads heavy operations to the system kernel and continues to process other requests.

7. Where Node.js is Used?

-  Web Servers
-  Real-time Applications (Chat, Gaming)
-  RESTful APIs
-  Streaming Applications
-  Serverless Functions
-  Microservices Architecture

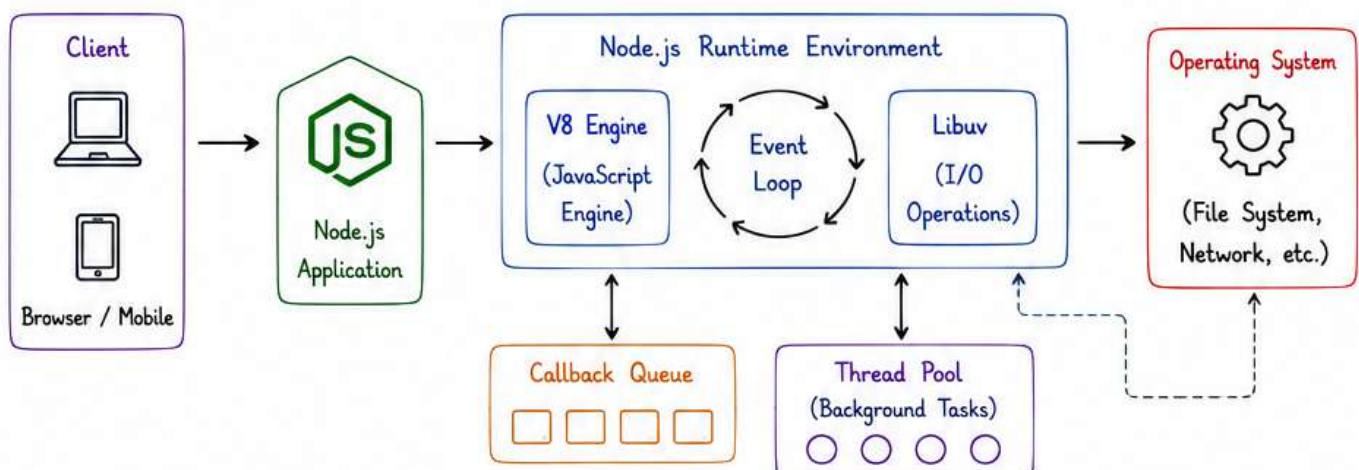
8. Advantages

- ✓ High Performance
- ✓ Scalable
- ✓ Rich Ecosystem (NPM)
- ✓ Full JavaScript Stack
- ✓ Great for Real-time Apps
- ✓ Lightweight & Fast

9. Limitations

- ✗ Not ideal for CPU intensive tasks
- ✗ Callback Hell (in old code)
- ✗ Single Thread Limitation
- ✗ Unstable APIs (rarely)

10. Node.js Architecture (High Level)



Interview Tip:

Node.js is preferred for **I/O bound** and **real-time** applications, not CPU intensive tasks.

Node.js Event Loop & Asynchronous Programming

★ One of the most asked interview topics!

Event Loop Diagram

1. Call Stack

- It is a stack data structure.
- It stores the current execution context.
- JS is single threaded, so only one operation goes into call stack at a time.

2. Callback Queue

- Stores callback functions ready to be executed.
- Also called Task Queue.

3. Microtask Queue

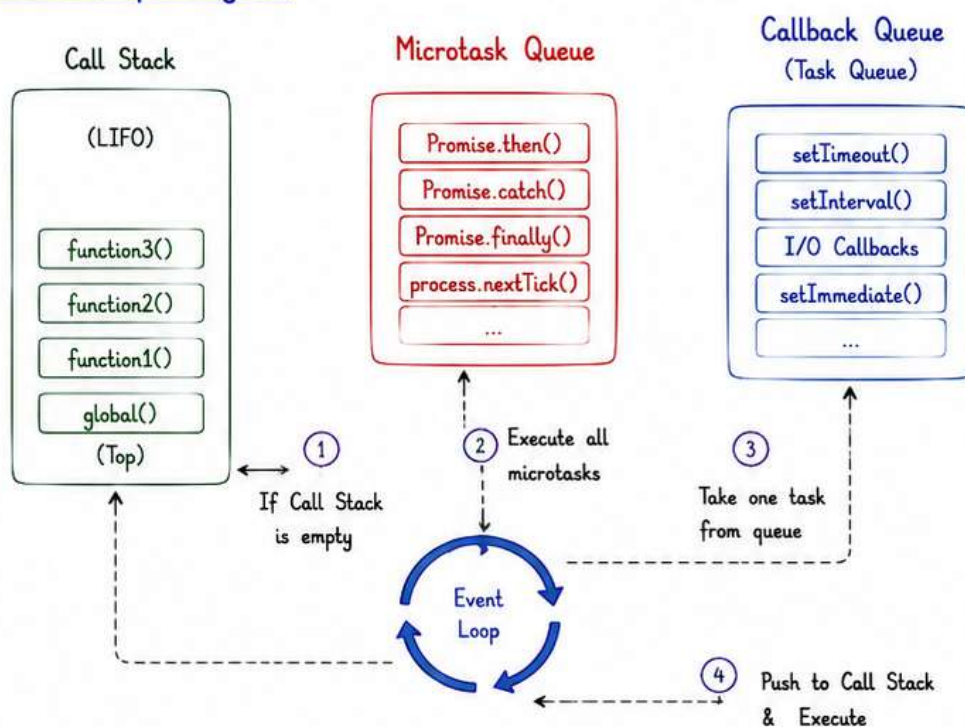
- Stores microtasks like Promises, process.nextTick().
- Has higher priority than Callback Queue.

4. Event Loop

- Continuously checks call stack.
- If call stack is empty:
 - 1) Executes all microtasks.
 - 2) Then takes one task from callback queue and pushes it to call stack.

5. Promises

- Handled in Microtask Queue.
- .then(), .catch(), .finally() are executed as microtasks.



6. setTimeout

- Executes after minimum delay.
- Goes to Callback Queue after delay is over.
- Not a precise timer.

7. setImmediate

- Executes immediately after I/O events.
- Goes to Callback Queue.
- Runs before setTimeout in most cases.

8. process.nextTick

- Executes before any other microtasks.
- Added to Microtask Queue with highest priority.
- Use it carefully (can block the event loop).

9. async / await

- Built on top of Promises.
- Makes async code look synchronous.
- await pauses execution until Promise is resolved.

Example Flow

```
console.log("Start"); // 1
setTimeout(() => console.log("Timeout"), 0); // 2
Promise.resolve().then(() => console.log("Promise")); // 3
process.nextTick(() => console.log("nextTick")); // 4
setImmediate(() => console.log("setImmediate")); // 5
console.log("End"); // 6
```

Output Order

- | | | |
|-----------------|---|--------------------------------|
| 1. Start | } | Synchronous (Call Stack) |
| 2. End | | |
| 3. nextTick | → | (Microtask - highest priority) |
| 4. Promise | → | (Microtask) |
| 5. setImmediate | → | (Callback Queue) |
| 6. Timeout | → | (Callback Queue - after delay) |



Interview Tip:

Understand the priority: nextTick > Promises (Microtask) > setImmediate > setTimeout

Node.js Core Modules

★ Built-in modules that come with Node.js ★



Interview Tip

Core modules are written in C/C++ and highly optimized.

1. fs (File System)

- Interact with file system.
- Read, write, update, delete files.

```
const fs = require('fs');
// Read File
fs.readFile('hello.txt', 'utf8',
  (err, data) => {
    if(err) throw err;
    console.log(data);
  });
```

2. path

- Utilities for working with file and directory paths.

```
const path = require('path');
console.log(path.join(__dirname,
  'folder', 'file.txt'));
console.log(path.basename(__filename));
console.log(path.extname(__filename));
```

3. http

- Create HTTP server & make HTTP requests.

```
const http = require('http');
const server = http.createServer
  ( (req, res) => {
    res.writeHead(200, {'Content-Type':
      'text/plain'});
    res.end('Hello Node.js!');
  });
server.listen(3000);
```

4. https

- Same as http but with SSL/TLS (secure).

```
const https = require('https');
https.get('https://example.com',
  (res) => {
    console.log(res.statusCode);
  });
```

5. os

- Provides operating system related information.

```
const os = require('os');
console.log(os.platform());
console.log(os.cpus().length);
console.log(os.totalmem());
console.log(os.freemem());
```

6. crypto

- Provides cryptographic functionality like hashing, encryption, etc.

```
const crypto = require('crypto');
const hash = crypto.createHash('sha256')
  .update('hello')
  .digest('hex');
console.log(hash);
```

7. stream

- Handle streaming data efficiently.
- Useful for large files.

```
const fs = require('fs');
const readStream = fs.createReadStream
  ('input.txt');
const writeStream = fs.createWriteStream
  ('output.txt');
readStream.pipe(writeStream);
```

8. buffer

- Handle binary data directly.
- Useful for working with raw data.

```
const buf = Buffer.from('Node.js');
console.log(buf); // <Buffer ...>
console.log(buf.toString()); // Node.js
console.log(buf.length); // length
```

9. events (EventEmitter)

- Create, fire and listen for events.
- Used for event driven programming.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('greet', (name) => {
  console.log('Hello ' + name);
});
emitter.emit('greet', 'Abhi');
```

10. process

- Provides information about & control over the Node.js process.

```
console.log(process.pid); // Process ID
console.log(process.platform); // Platform
console.log(process.env); // Environment variables
process.exit(0); // Exit process
```

Other Important Core Modules

url
querystring
util
events
zlib
timers
child_process

Parse and format URL.
Parse URL query string.
Utility functions like format, inspect, etc.
(Already covered) Event handling.
Compression / decompression.
setTimeout, setInterval (global but part of timers).
Spawn new processes.



Key Takeaway

Core modules are the **foundation** of Node.js. Master them to build **robust and efficient** backend applications.

Express.js Complete Guide

★ The Most Popular Node.js Framework ★

@abhi_techhub



Interview Tip

Express.js is a minimal and flexible framework for building web applications and APIs.

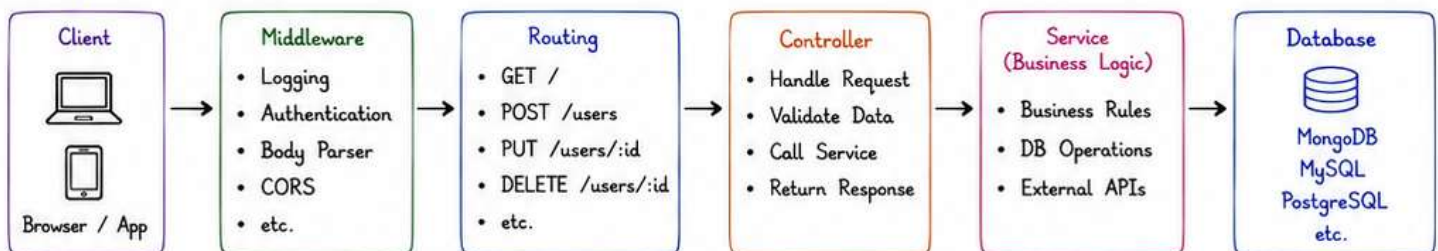
1. What is Express.js?

Express.js is a fast, unopinionated, minimalist web framework for Node.js. It helps us build web apps and APIs easily.

2. Why Express.js?

- ✓ Minimal and flexible
- ✓ Easy to learn and integrate
- ✓ Powerful routing
- ✓ Middleware support
- ✓ Huge ecosystem (NPM)
- ✓ Perfect for REST APIs

3. Express Architecture



4. Routing

```

const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Home'));
app.post('/users', (req, res) => res.send('User Created'));
app.put('/users/:id', (req, res) => res.send('User Updated'));
app.delete('/users/:id', (req, res) => res.send('User Deleted'));
  
```

5. Middleware

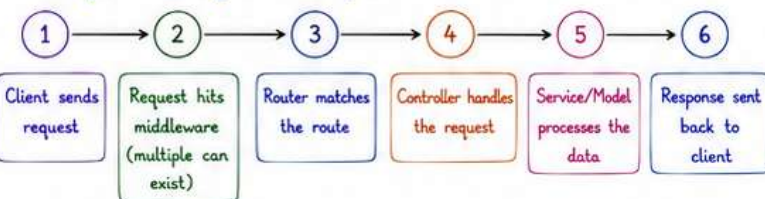
- Functions that have access to req, res, and next() in the request-response cycle.
- Can execute code, modify request/response, end cycle or call next().

```

// Built-in Middleware
app.use(express.json()); // Parse JSON body
app.use(express.urlencoded({ extended: true }));
app.use(express.static('public')); // Serve static files

// Custom Middleware
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};
app.use(logger);
  
```

6. Request Lifecycle in Express



7. Error Handling

Use error handling middleware with 4 parameters (err, req, res, next) at the end.

```

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({
    success: false,
    message: 'Something went wrong!'
  });
});
  
```

8. Folder Structure (Best Practice)

```

my-app/
├── src/
│   ├── config/      # DB, env, config
│   ├── controllers/ # Request handlers
│   ├── routes/      # Route definitions
│   ├── services/    # Business logic
│   ├── models/      # DB models
│   ├── middlewares/ # Custom middlewares
│   ├── utils/       # Helper functions
│   └── app.js        # Express app setup
├── .env
├── package.json
└── server.js        # Entry point
  
```

9. REST API Design

Method	Endpoint	Description	Example
GET	/users	Get all users	GET /users
GET	/users/:id	Get user by id	GET /users/1
POST	/users	Create new user	POST /users
PUT	/users/:id	Update user	PUT /users/1
PATCH	/users/:id	Partial update	PATCH /users/1
DELETE	/users/:id	Delete user	DELETE /users/1

Best Practices

- ✓ Keep routes clean and small
- ✓ Use meaningful route names
- ✓ Validate input data
- ✓ Use proper status codes
- ✓ Handle errors gracefully
- ✓ Keep business logic in services
- ✓ Don't repeat yourself (DRY)
- ✓ Environment based configs
- ✓ Use .env for secrets
- ✓ Follow consistent folder structure



Summary: Express.js simplifies backend development in Node.js. Master middleware, routing and error handling.



★ Secure your APIs and protect your application ★

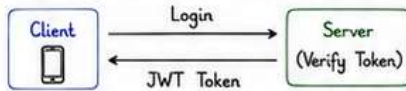


Interview Tip

Always authenticate, authorize and validate before processing request.

1. JWT (JSON Web Token)

- Stateless authentication mechanism.
- Token is signed using a secret or public/private key.
- Sent by client in Authorization header.



```
// Generate Token
const token = jwt.sign({ id: user.id },
  process.env.JWT_SECRET, { expiresIn: '7d' });
// Verify Token
const decoded = jwt.verify(token,
  process.env.JWT_SECRET);
```

2. Cookies

- Small data stored in browser.
- Sent automatically with every request.
- Can be used to store session or JWT.
- Use httpOnly and secure flag.

```
res.cookie('token', token, {
  httpOnly: true,
  secure: true,
  sameSite: 'strict',
  maxAge: 7 * 24 * 60 * 60 * 1000
});
```

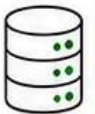


3. Sessions

- Stores session data on server.
- Session ID stored in cookie.
- More secure but stateful.
- Requires store like Redis / DB.

```
req.session.userId = user.id;
req.session.isLoggedIn = true;

// Access
req.session.userId;
```



4. OAuth 2.0

- Authorization framework.
- Allows users to login using Google, GitHub, Facebook, etc.
- Uses access token.



```
passport.use(new GoogleStrategy({
  clientID: GOOGLE_ID,
  clientSecret: GOOGLE_SECRET,
  callbackURL: '/auth/google/callback'
}, verifyCallback));
```

5. bcrypt (Password Hashing)

- Never store passwords in plain text.
- Use bcrypt to hash passwords.
- Strong and slow hashing algorithm.



```
const bcrypt = require('bcrypt');
// Hash Password
const hash = await bcrypt.hash(password, 10);
// Compare Password
const isMatch = await bcrypt.compare(
  password, hash);
```

6. Helmet (Secure Headers)

- Helps set various HTTP headers to secure app.
- Protects against common attacks.



```
const helmet = require('helmet');
app.use(helmet());

// It sets headers like:
// X-Content-Type-Options, X-Frame-Options,
// X-XSS-Protection, Strict-Transport-Security
```

7. CORS (Cross-Origin Resource Sharing)

- Allows or restricts resources on different origin.
- Important for APIs.



```
const cors = require('cors');
app.use(cors({
  origin: 'https://your-frontend.com',
  methods: ['GET', 'POST', 'PUT', 'DELETE']
}));
```

8. CSRF (Cross-Site Request Forgery)

- Attack that tricks user to perform unwanted action.
- Mitigation: Use CSRF token.



```
const csrf = require('csrf');
app.use(csrf({ cookie: true }));
// Send token to client
res.render('form', { csrfToken: req.csrfToken() });
// Validate token in requests
```

9. XSS (Cross-Site Scripting)

- Inject malicious scripts in web pages.
- Mitigation: Sanitize user input and escape output.



```
// Use libraries like 'xss-clean'
const xss = require('xss-clean');
app.use(xss());
// Or escape output using template engines like EJS, Pug
```

10. Rate Limiting

- Prevents abuse by limiting number of requests.
- Protects from brute force & DDoS.



```
const rateLimit = require('express-rate-limit');
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 min
  max: 100 // max 100 requests
});
app.use('/api/', limiter);
```

JWT vs Session

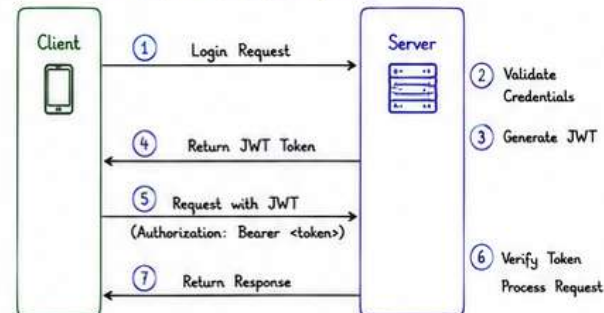
Feature	JWT	Session
Storage	Client (Browser)	Server (DB/Redis)
State	Stateless	Stateful
Scalability	Highly Scalable	Less Scalable
Revocation	Difficult	Easy (Destroy session)
Use Case	APIs, Microservices	Web Applications
Security	More (if implemented right)	More Secure (server side)

Security Checklist

- ✓ Use HTTPS everywhere
- ✓ Validate and sanitize all inputs
- ✓ Hash passwords using bcrypt
- ✓ Use JWT with short expiration
- ✓ Store secrets in environment variables
- ✓ Use Helmet for secure headers
- ✓ Enable CORS with specific origin
- ✓ Add Rate Limiting
- ✓ Protect against CSRF and XSS
- ✓ Keep dependencies updated
- ✓ Use least privilege access
- ✓ Log and monitor suspicious activities



Authentication Flow (JWT Based)



Key Takeaway: Authentication identifies the user. Authorization defines what user can access. Security is not a feature, it is a mindset!



Node.js Database & ORM

★ Store, manage and interact with data efficiently ★



Interview Tip

Know the database, ORM, connections, transactions and performance concepts.

1. Popular Databases



MongoDB - NoSQL (Document Based)



PostgreSQL - SQL (Relational)



MySQL - SQL (Relational)



Others - SQLite, MariaDB, Oracle

2. ORM / ODM in Node.js

ORM (Object Relational Mapping) → For SQL DBs

- Prisma
- Sequelize

ODM (Object Document Mapping) → For NoSQL DBs

- Mongoose (for MongoDB)

3. Database Connection

```
// Example using Mongoose (MongoDB)
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/myapp', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('DB Connected'))
.catch(err => console.log('DB Error:', err));
```

4. Mongoose (MongoDB ODM)

```
// Define Schema
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});

// Create Model
const User = mongoose.model('User', userSchema);

// Insert Document
const user = new User({ name: 'Abhi', email: 'abhi@mail.com',
age: 22 });
await user.save();

// Find Documents
const users = await User.find({ age: { $gt: 18 } });

// Update Document
await User.updateOne({ email: 'abhi@mail.com' }, { age: 23 });

// Delete Document
await User.deleteOne({ email: 'abhi@mail.com' });
```

5. Prisma (SQL ORM)

```
// Initialize Prisma
npx prisma init

// Example: Find Users
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

const users = await prisma.user.findMany({
  where: { age: { gt: 18 } }
});

// Create User
const newUser = await prisma.user.create({
  data: {
    name: 'Abhi',
    email: 'abhi@mail.com',
    age: 22
  }
});
```

6. Sequelize (SQL ORM)

```
const { Sequelize, DataTypes } = require('sequelize');
const sequelize = new Sequelize('mydb', 'root', 'password', {
  host: 'localhost',
  dialect: 'mysql'
});

const User = sequelize.define('User', {
  name: DataTypes.STRING,
  email: DataTypes.STRING,
  age: DataTypes.INTEGER
});

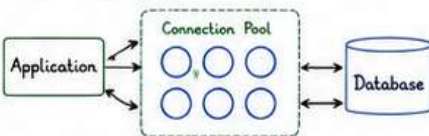
// Sync Model
await sequelize.sync();

// Create
await User.create({ name: 'Abhi', email: 'abhi@mail.com', age: 22 });

// Find
const users = await User.findAll();
```

7. Connection Pooling

- Maintain a pool of connections.
- Reuse connections to improve performance.
- Prevents overhead of opening new connection for every request.



```
// In Mongoose (poolSize)
mongoose.connect(uri, { maxPoolSize: 10 });
```

8. Transactions (SQL)

- Group multiple operations into a single unit.
- Either all succeed or all fail.

```
// Prisma Example
await prisma.$transaction([
  prisma.user.create({ data: { name: 'A' } }),
  prisma.post.create({ data: { title: 'Hello',
    userId: 1 } })
]);

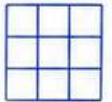
// Sequelize Example
const t = await sequelize.transaction();
try {
  await User.create({ name: 'A' }, { transaction: t });
  await Post.create({ title: 'Hello' }, { transaction: t });
  await t.commit();
} catch (err) {
  await t.rollback();
}
```

9. Indexing

- Improves query performance.
- Reduces search time.
- Use indexes on fields used in filtering, sorting, joining.

```
// MongoDB
db.users.createIndex({ email: 1 });
```

```
// SQL (PostgreSQL / MySQL)
CREATE INDEX idx_email
ON users (email);
```



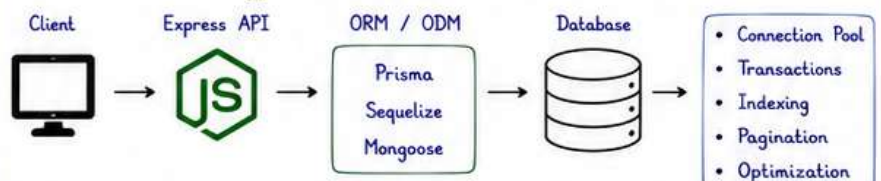
10. Pagination

- Limit results and load in pages.
- Improves performance & user experience.

```
// MongoDB
const page = 2, limit = 10;
const data = await User.find()
  .skip((page-1)*limit)
  .limit(limit);

// SQL (Prisma)
const data = await prisma.user.findMany({
  skip: (page-1)*limit,
  take: limit
});
```

Typical Architecture



11. ORM / ODM Comparison

Tool	Type	Database	Language	Best For	Key Features
Mongoose	ODM	MongoDB	JS/TS	NoSQL Apps	Schemas, Middleware, Aggregation, Validation
Prisma	ORM	PostgreSQL, MySQL, SQLite, etc.	JS/TS	Modern SQL Apps	Type Safety, Auto Migrations, Relations, Studio
Sequelize	ORM	PostgreSQL, MySQL, MariaDB, SQLite, MSSQL	JS/TS	SQL Apps	Associations, Migrations, Transactions



Key Takeaway: Choose the right database and ORM/ODM based on your use case.
Focus on performance, indexing, transactions and clean data access patterns.





Handle more users, more requests and grow your application

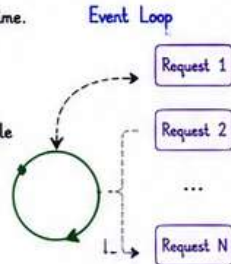


Interview Tip

Node.js can scale horizontally (add more machines) since it is single threaded per instance.

1. Node.js is Single Threaded (Per Process)

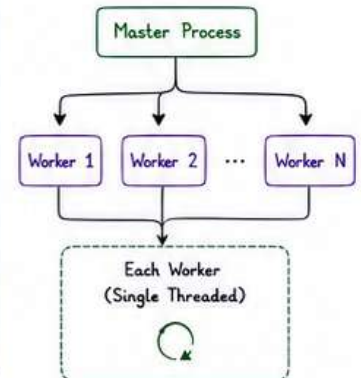
- One thread handles one request at a time.
- Uses non-blocking I/O to handle thousands of concurrent requests.
- For CPU-intensive tasks $\Rightarrow$ use multiple processes / threads.



2. Cluster Module (Multi-Process)

- Creates multiple Node.js processes.
- Each process has its own memory.
- Useful to utilize multi-core CPU.

```
const cluster = require('cluster');
const os = require('os');
if (cluster.isMaster) {
  const numCPUs = os.cpus().length;
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
} else {
  // Your app code
}
```



3. PM2 (Process Manager)

- Production ready process manager.
- Start, stop, restart, monitor apps.
- Auto restart on crash.
- Load balancing in Cluster mode.

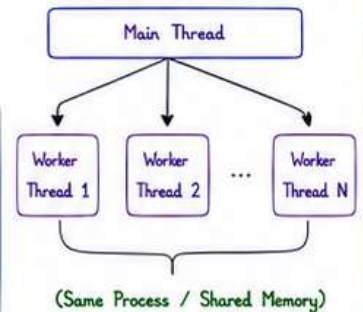


```
# Install
npm install -g pm2
# Start App
pm2 start app.js -i max    # cluster mode
pm2 list                   # list all apps
pm2 monitor                # monitoring
pm2 restart all            # restart
pm2 logs                   # logs
```

4. Worker Threads

- Runs multiple threads in same process.
- Shares memory, faster communication.
- Good for CPU-intensive tasks.

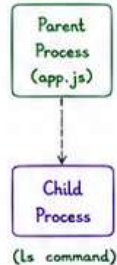
```
const { Worker } = require('worker_threads');
const worker = new Worker('./worker.js');
worker.on('message', (data) => {
  console.log(data);
});
worker.postMessage({ task: 'start' });
```



5. Child Process

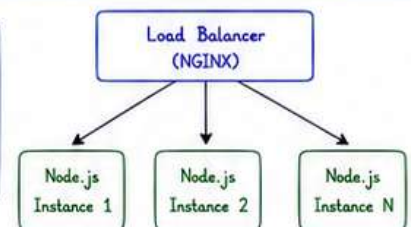
- Creates a separate process.
- Does not share memory.
- Used to run shell commands, heavy tasks.

```
const { spawn } = require('child_process');
const process = spawn('ls', ['-la']);
process.stdout.on('data', (data) => {
  console.log(`output: ${data}`);
});
```



6. Load Balancer

- Distributes incoming traffic across multiple Node.js instances.
- Improves availability & scalability.
- Examples: NGINX, HAProxy, AWS ALB



7. Redis (Caching & Session Store)

- In-memory data store.
- Used for caching, session storage, rate limiting, pub/sub etc.



```
const redis = require('redis');
const client = redis.createClient();
client.set('key', 'value');
client.get('key', (err, data) => {
  console.log(data);
});
```

Use Cases:

- ✓ Caching
- ✓ Session Store
- ✓ Rate Limiting
- ✓ Leaderboard
- ✓ Pub/Sub

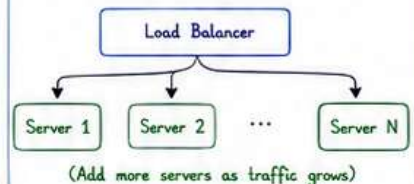
8. Message Queue

- Decouple services.
- Handle async jobs.
- Improve reliability.
- Examples: RabbitMQ, Kafka, BullMQ



9. Horizontal Scaling

- Add more machines/instances.
- Distribute traffic using Load Balancer.
- Best way to handle high traffic.



10. Deployment Options

Docker



- Containerize app
- Consistent environment
- Easy to deploy

```
docker build
docker run -p 3000:3000 app
```

Kubernetes



- Orchestration
- Auto scaling
- Self healing
- Rolling updates

Cloud Platforms



- Managed services
- Auto scaling
- High availability

PaaS (Heroku / Render / Railway)



- Easy deployment
- No server management
- Auto scaling

git push heroku main



Key Takeaway:

Use Cluster/PM2 for multi-core, Redis for cache, Queues for async jobs, Load Balancer for distribution and Horizontal Scaling for growth!



Node.js Streams & Performance

★ Write efficient, scalable and high-performance Node.js apps ★



Interview Tip

Use Streams, Caching and Good Practices to build fast and scalable apps.

1. Streams

- Streams are used to read/write data piece by piece.
- Very useful for large files or real-time data.
- Types: Readable, Writable, Duplex, Transform.



```
const fs = require('fs');
const readStream = fs.createReadStream('input.txt');
const writeStream = fs.createWriteStream('output.txt');
readStream.pipe(writeStream); // piping
```

2. Buffers

- Buffer is used to handle binary data.
- Similar to arrays, but contains raw data.
- Fixed size.

Buffer (5 bytes)

72	101	108	108	111
H	e	l	l	o

```
const buf = Buffer.from('Hello');
console.log(buf); // <Buffer 48 65 6c 6c 6f>
console.log(buf.toString()); // Hello
```

3. Piping

- Piping connects readable stream to writable stream.
- Automatically handles data flow and errors.



```
fs.createReadStream('bigfile.zip')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('bigfile.zip.gz'));

// read -> compress -> write
```

4. File Upload (Using Streams)

- Store incoming file directly to disk without using memory.

```
const http = require('http');
const fs = require('fs');
const server = http.createServer((req, res) => {
  if (req.method === 'POST') {
    const fileStream = fs.createWriteStream('upload.dat');
    req.pipe(fileStream);
    fileStream.on('finish', () => res.end('Upload Done'));
  }
});
server.listen(3000);
```

5. Compression

- Compress response to reduce size.
- Improves performance.

```
const express = require('express');
const compression = require('compression');
const app = express();

app.use(compression());

app.get('/', (req, res) => {
  res.send('Hello Abhi!');
});

app.listen(3000);
```

6. Caching

- Store frequently used data. ✓ In-Memory Cache (node-cache)
- Reduces database calls. ✓ Redis
- Improves response time. ✓ Memcached

Popular Caching Options

```
const NodeCache = require('node-cache');
const cache = new NodeCache({ stdTTL: 60 }); // 60 sec
cache.set('user:1', { id: 1, name: 'Abhi' });
const user = cache.get('user:1');
console.log(user);
```

7. Memory Leaks

- Memory leaks make app slow and may crash.
- Causes:
 - ✓ Global variables
 - ✓ Unclosed connections / listeners
 - ✓ Unbounded caches
 - ✓ Huge arrays / objects
 - ✓ Closures holding references

Tips

- Use tools (heapdump, clinic.js)
- Close connections
- Remove unused listeners
- Monitor memory usage

8. Garbage Collection (GC)

- Automatically reclaims memory not used by the app.
- Node.js uses V8 GC (Mark & Sweep, Scavenge).
- Write clean code to help GC.

Best Practices for GC

- ✓ Avoid global variables
- ✓ Release references
- ✓ Use streams for large data
- ✓ Avoid memory leaks

9. EventEmitter

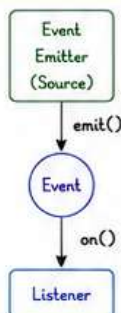
- Core module to handle events.
- Used in many Node.js modules.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('greet', (name) => {
  console.log('Hello ${name}');
});

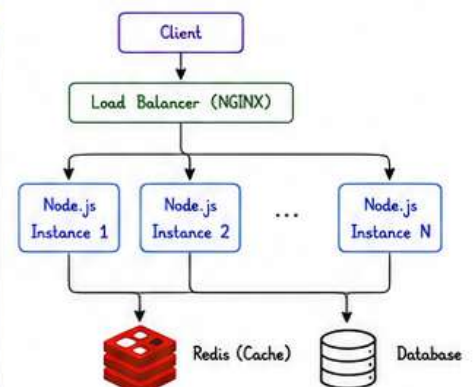
emitter.emit('greet', 'Abhi');

// Hello Abhi
```



10. Performance Optimization Tips

- ✓ Use Streams for large data
- ✓ Enable Compression
- ✓ Cache frequent data
- ✓ Use Connection Pooling
- ✓ Avoid synchronous/blocking code
- ✓ Use Worker Threads for CPU heavy tasks
- ✓ Use async/await and Promises
- ✓ Minimize dependencies
- ✓ Keep Node.js updated
- ✓ Use proper logging (avoid console.log in prod)
- ✓ Monitor with PM2 / Prometheus
- ✓ Use cluster mode / load balancing



Key Takeaway: Use Streams to handle large data, Cache to reduce load, and follow Best Practices to write fast and scalable Node.js applications.



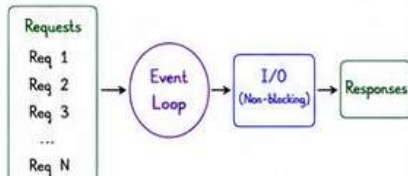


Interview Tip

Answer in points, give examples from real world & mention performance impact wherever possible.

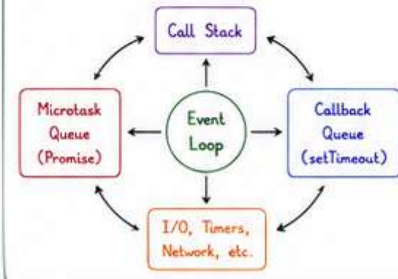
1. Why is Node.js Single-threaded?

- Node.js uses single thread with Event Loop.
- It is non-blocking I/O, so it can handle thousands of concurrent requests.
- CPU-intensive tasks can block the event loop, so use Worker Threads/Cluster.



2. Explain Event Loop in Node.js?

- Event Loop is the heart of Node.js.
- It allows non-blocking I/O operations.
- It continuously checks Call Stack, Microtask Queue and Callback Queue.



3. process.nextTick() vs setImmediate()

process.nextTick()	setImmediate()
1. Goes to Microtask Queue	1. Goes to Check Phase (Callback Queue)
2. Executes before setImmediate()	2. Executes after I/O events
3. Can starve the Event Loop if used recursively	3. Depends on Event Loop phases
<pre>process.nextTick(() => { console.log('nextTick'); });</pre>	<pre>setImmediate(() => { console.log('immediate'); });</pre>

4. Streams vs Buffers?

Streams	Buffers
- Used to read/write large data piece by piece. - Memory efficient. - Examples: fs.createReadStream, fs.createWriteStream	- Used to handle small binary data. - Stores data in fixed size chunks. - Example: Buffer.from()
Use Streams for large files, Buffers for small data.	

5. CommonJS vs ES Modules?

	CommonJS (CJS)	ES Modules (ESM)
Syntax	require(), module.exports	import, export
Loading	Synchronous	Asynchronous
Default Support	Yes	Yes
Use	Older Node.js applications	Modern Node.js applications
File Ext.	.js	.mjs or "type": "module" in package.json

6. JWT vs Session?

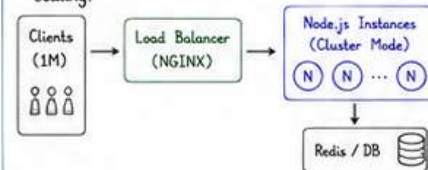
JWT	Session
- Stateless - Token stored in client (localStorage, cookie) - Scalable - Good for APIs & microservices	- Stateful - Data stored on server (usually in Redis/DB) - Less scalable - Good for traditional web apps
JWT = Scalability Session = Security (easy to revoke)	

7. Child Process vs Worker Threads?

Child Process	Worker Threads
- Creates separate process. - Has its own memory. - Good for isolation. - More overhead.	- Runs in same process. - Shares memory (with limitations). - Good for CPU-intensive tasks. - Less overhead.
Use Child Process for isolation, Worker Threads for performance.	

8. How does Node.js handle 1 Million Concurrent Connections?

- Node.js uses non-blocking I/O and Event Loop.
- Uses OS features like epoll (Linux) or kqueue (Mac) for handling connections.
- Minimal memory per connection.
- Use Cluster / Load Balancer for horizontal scaling.



9. Why use Express.js?

- Minimal and flexible framework.
- Huge ecosystem (middlewares, plugins).
- Easy routing and middleware support.
- Fast development.
- Production ready.

```
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send('Hello'));
app.listen(3000);
```

10. Middleware in Node.js / Express?

- Functions that have access to req, res and next().
- Can execute code, modify request/response, end cycle or call next().
- Types: Application, Router, Error handling, Built-in, Third-party.

```
app.use((req, res, next) => {
  console.log('Time:', Date.now());
  next(); // pass to next middleware
});
```

11. Error Handling in Express?

- Use try/catch for sync code.
- Use next(err) for async errors.
- Centralized error handling middleware.

```
app.get('/', async (req, res, next) => {
  try {
    throw new Error('Something went wrong');
  } catch (err) {
    next(err);
  }
});

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Internal Server Error' });
});
```

12. Environment Variables in Node.js?

- Keep sensitive data outside code.
- Use .env file and dotenv package.
- Access via process.env.VARIABLE_NAME

```
require('dotenv').config();
const dbHost = process.env.DB_HOST;
const dbUser = process.env.DB_USER;
console.log(dbHost, dbUser);
```

Best Practices

- ✓ Validate env variables
- ✓ Do not commit .env to Git
- ✓ Use different .env for dev, staging, prod





Write clean, maintainable and production-ready Node.js applications



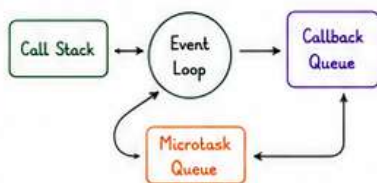
Interview Tip

Focus on concepts, trade-offs and real-world use cases.

Quality over quantity!

1. Asynchronous Nature

- Node.js is non-blocking and event-driven.
- Uses Event Loop, Call Stack, Callback Queue, Microtask Queue.
- Helps handle high concurrency.



2. Error Handling

- Always handle errors in async code.
- Use try-catch and next(err) in Express.
- Centralized error handling middleware.

```

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    message: err.message || 'Something went wrong'
  });
});

```

3. Middleware

- Functions that have access to req, res, next().
- Executed in the order they are defined.
- Types: Built-in, Third-party, Custom.

```

const logger = (req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
};
app.use(logger);

```

4. Environment Variables

- Store sensitive data in .env file.
- Access using process.env.
- Use dotenv package.

```

require('dotenv').config();
const PORT = process.env.PORT;
const DB_USER = process.env.DB_USER;
console.log(PORT);

```

Best Practices

- ✓ Never commit .env file
- ✓ Use different .env for dev, staging, prod

5. Modules in Node.js

- Node.js uses CommonJS (require, module.exports) by default.
- ES Modules (import/export) are also supported.

CommonJS	ES Modules
<pre> // math.js module.exports = { add: (a, b) => a + b }; </pre>	<pre> // math.js export const add = (a, b) => a + b; </pre>
<pre> // index.js const math = require('./math'); console.log(math.add(2, 3)); </pre>	<pre> // index.js import { add } from './math.js'; console.log(add(2, 3)); </pre>

6. File System (FS)

- Built-in module to work with files.
- Supports async (non-blocking) operations.

```

const fs = require('fs');
// Read File
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
// Write File
fs.writeFile('file.txt', 'Hello Node.js', err => {
  if (err) throw err;
  console.log('File written');
});

```

7. Security Best Practices

- ✓ Validate and sanitize all inputs
- ✓ Use Helmet for secure headers
- ✓ Prevent SQL Injection & XSS
- ✓ Use bcrypt for password hashing
- ✓ Limit rate with express-rate-limit
- ✓ Keep dependencies updated



8. Database Integration

- Popular: MongoDB (NoSQL), PostgreSQL, MySQL.
- Use ODM/ORM for better abstraction.
- Example with Mongoose (MongoDB):

```

const mongoose = require('mongoose');
mongoose.connect(process.env.MONGO_URL);
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});
const User = mongoose.model('User', userSchema);

```

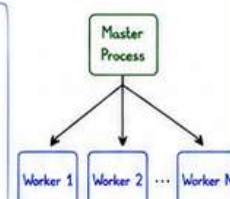
9. Clustering

- Utilize multi-core CPU.
- Improves performance and reliability.

```

const cluster = require('cluster');
const os = require('os');
if (cluster.isMaster) {
  const numCPUs = os.cpus().length;
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
} else {
  require('./app');
}

```



10. Caching

- Reduces DB calls and improves response time.
- Use: Redis, in-memory cache.

```

const redis = require('redis');
const client = redis.createClient();
await client.connect();
await client.set('key', 'value', { EX: 60 });
const data = await client.get('key');

```

11. Logging

- Use logging libraries for better logs.
- Popular: Winston, Pino, Morgan.
- Log levels: info, warn, error, debug.

```

const winston = require('winston');
const logger = winston.createLogger({
  level: 'info',
  transports: [new winston.transports.Console()]
});
logger.info('Server started 🚀');

```

12. PM2 (Process Manager)

- Keeps app alive, auto restart, load balancing.
- Monitor CPU, memory, logs.
- Commands:

```

pm2 start app.js
pm2 list
pm2 restart all
pm2 logs

```

PM2

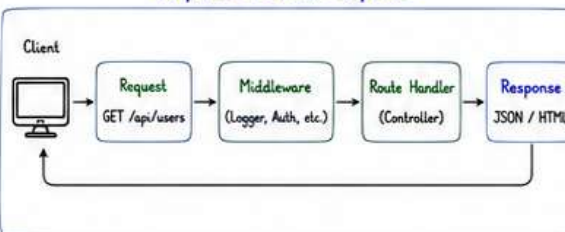
Node.js Application Structure (Example)

```

my-app/
├── config/           # Configuration files
├── controllers/      # Route handlers
├── models/           # DB models
├── routes/           # API routes
├── middleware/       # Custom middleware
├── services/         # Business logic
├── utils/            # Helper functions
├── app.js            # Entry point
└── .env              # Environment variables

```

Request Flow in Express



Performance Tips

- ✓ Use Streams for large files
- ✓ Enable Compression
- ✓ Cache frequently used data
- ✓ Use Connection Pooling
- ✓ Avoid blocking code
- ✓ Use Worker Threads for CPU intensive tasks



Key Takeaway: Understand the core, follow best practices, and keep learning by building real projects.

Clean code + Good architecture = Scalable & maintainable applications! 🚀

